

Data de Execução: 23/03/2025

Planejamento Didático: Aplicações Práticas Avançadas

Semana 7 - Aula 1

Tópico Principal: Segurança em Back-end

Subtítulo/Tema Específico: Implementação de Autenticação e Autorização

Código da Aula: [SIS]ANO2-S7-A1

Objetivos da Aula

1. Compreender a distinção teórica e prática entre autenticação e autorização em sistemas back-end.
2. Implementar fluxos básicos de autenticação em aplicações web.

Recursos Adicionais

- Computador com ambiente de desenvolvimento instalado (Node.js/Python, IDE como VS Code).
- Acesso à internet para consulta de documentação (ex: JWT.io).
- Caderno para esboços de arquitetura de segurança.

Exposição Detalhada do Conteúdo

- **Referência do Slide:** [Construindo o conceito - Sistema de autenticação e autorização]
 - **Definição:** A autenticação é o processo de confirmar a identidade de um usuário, geralmente através de credenciais (login/senha). Já a autorização é o processo que determina se um usuário, após autenticado, tem permissão para acessar recursos ou realizar ações específicas.
 - **Aprofundamento (Busca Externa):** [Informação complementar via busca externa] A autenticação costuma utilizar protocolos como OAuth2 ou OpenID Connect para delegação de acesso seguro. A autorização é frequentemente implementada via RBAC (Role-Based Access Control) ou ABAC (Attribute-Based Access Control), garantindo o princípio do privilégio mínimo.
 - **Exemplo Prático:** Em um aplicativo de caronas, a autenticação confirma que o motorista é quem ele diz ser através do login; a autorização garante que apenas o "motorista" possa alterar as rotas de viagem, enquanto o "passageiro" tenha permissão apenas para visualizar e pagar a corrida.

Fechamento da Aula

- **Resumo:**
 1. Autenticação valida a identidade (quem você é).
 2. Autorização valida permissões (o que você pode fazer).
 3. JWTs são fundamentais para manter estados de segurança de forma leve e escalável.
- **Curadoria Digital:** [Dicionário do Programador - JWT](#)
- **Bibliografia:**
 1. LEPOW, D.; ALASHI, E.; GIMBLETT, B. *Autenticação e autorização no Gerenciamento de API do Azure*. Microsoft Learn, 2023.

2. SHARABI, K.; et al. *Visão geral da autenticação e autorização do back-end*. Microsoft Learn, 2024.

Semana 7 - Aula 2

Tópico Principal: Integração de Sistemas

Subtítulo/Tema Específico: Integração com APIs Externas

Código da Aula: [SIS]ANO2-S7-A2

Objetivos da Aula

1. Integrar aplicações back-end com serviços de terceiros via APIs externas.
2. Implementar estratégias de segurança e resiliência na comunicação entre APIs.

Recursos Adicionais

- Ferramenta de testes de API (ex: Postman ou Insomnia).
- Documentação de uma API pública (ex: JSONPlaceholder ou ViaCEP).

Exposição Detalhada do Conteúdo

- **Referência do Slide:** [Integração com API externas]
 - **Definição:** É o uso de serviços de terceiros para agregar funcionalidades ao back-end, como processamento de pagamentos, consulta de endereços ou mapas, sem necessidade de construir a lógica do zero.
 - **Aprofundamento (Busca Externa):** [Informação complementar via busca externa] Ao integrar APIs, deve-se sempre implementar o tratamento de erros (ex: HTTP 4xx, 5xx), uso de *timeouts* e *circuit breakers* para evitar que uma falha externa derrube todo o seu sistema. Além disso, segredos de API (*API Keys*) nunca devem ser expostos no código-fonte, utilizando variáveis de ambiente (.env).
 - **Exemplo Prático:** Uma loja virtual que integra a API dos Correios para calcular o frete automaticamente, usando a chave de autenticação do lojista para garantir que os dados de entrega sejam processados com segurança.

Fechamento da Aula

- **Resumo:**
 1. APIs externas estendem funcionalidades do sistema.
 2. Segurança é essencial no uso de tokens/chaves de API.
 3. Resiliência (tratamento de erros) é vital para a estabilidade da aplicação.
- **Curadoria Digital:** [Curso de APIs - Como consumir APIs](#) (Nota: Link exemplificativo de tutorial de integração).
- **Bibliografia:**
 1. NEWMAN, S. *Building Microservices*. O'Reilly Media, 2021.
 2. FIELDING, R. T. *Architectural Styles and the Design of Network-based Software Architectures*. Tese de Doutorado, 2000.

Semana 7 - Aula 3

Tópico Principal: Comunicação em Tempo Real

Subtítulo/Tema Específico: WebSockets

Código da Aula: [SIS]ANO2-S7-A3

Objetivos da Aula

1. Implementar comunicação bidirecional entre servidor e cliente.
2. Diferenciar o fluxo de requisições HTTP do fluxo persistente de WebSockets.

Recursos Adicionais

- Biblioteca de WebSockets (ex: Socket.io para Node.js).
- Console do navegador para inspeção de rede.

Exposição Detalhada do Conteúdo

- **Referência do Slide:** [Implementação de WebSockets para comunicação em tempo real]
 - **Definição:** Diferente do modelo HTTP (onde o cliente pede e o servidor responde), o WebSocket cria um canal de comunicação bidirecional e persistente, permitindo que o servidor envie dados ao cliente instantaneamente, sem o cliente solicitar.
 - **Aprofundamento (Busca Externa):** [Informação complementar via busca externa] WebSockets utilizam o protocolo `ws://` ou `wss://` (seguro). São ideais para chats, dashboards de ações em tempo real e jogos multiplayer. Deve-se ter atenção redobrada com a autenticação no handshake inicial da conexão WebSocket.
 - **Exemplo Prático:** Um sistema de monitoramento de servidor que alerta o administrador sobre picos de CPU instantaneamente em uma tela dashboard, sem precisar que o administrador atualize a página a cada segundo.

Fechamento da Aula

- **Resumo:**
 1. WebSockets permitem comunicação bidirecional e tempo real.
 2. A conexão é mantida aberta (persistente).
 3. Substitui a necessidade de "polling" (consultas repetitivas ao servidor).
- **Curadoria Digital:** [Tutorial de Socket.io \(Node.js\)](#)
- **Bibliografia:**
 1. FESER, J. *WebSockets: A Guide for Developers*. Packt Publishing, 2022.
 2. W3C. *The WebSocket API Specification*. W3C Recommendation, 2011.